

D. Y. Patil Agriculture and Technical University, Talsande, Kolhapur

Program: BTech – Computer Science and Engineering

Course Name: Computer Organization and Architecture

(Course Code: BTCSE022401)



Course Conducted

By

Mrs. Ashwini N. Sawant

Assistant Professor of CSE,

Department of Computer Science and Engineering,

School of Engineering and Technology,

D. Y. Patil Agriculture and Technical University, Talsande, Kolhapur.

Unit 6

Input / Output (I/O) Organization

Input / Output (I/O) Organization

It is the way a computer system **communicates with external devices** (like keyboard, printer, etc.).

1. External devices : are **physical parts (hardware)** that are connected to a computer from outside.

- They are **not part of the CPU internally**.
- They are connected **from outside** using cables, ports, or wireless and help us to:

Examples:

Keyboard → used to type → **Input device** -----**Give input**

Monitor → shows results → **Output device** -----**Get output**

Pen drive (USB) → stores data → **Storage device**-----**Store data**

I/O Module- It is a hardware part that works as a bridge between the CPU, memory, and external devices.

Need of I/O Module

- CPU **cannot directly communicate** with external devices because
- Devices are **slower**.
- They use **different data formats**.
- So, I/O Module helps in **managing communication**.

How it works

- CPU sends request
- I/O Module receives it
- Communicates with device
- Transfers data
- Sends response back to CPU

Example: Printing a document:

- CPU sends print command
- I/O Module receive it
- sends it to printer
- Printer prints
- I/O Module informs CPU

Functions of I/O Module

1. Control & Timing → *Manages data flow*

- The I/O module controls **when and how data is transferred.**

It makes sure:

- Data is sent **at the right time**
- CPU and device work in **proper sequence**

Example:

CPU sends data → I/O module ensures device is ready before sending

2. Communication → *Between CPU and devices*

- I/O module acts as a **bridge.**
- It Receives instructions from CPU
- Sends commands to devices
- Transfers data both ways

Example:

CPU → “Print this file”

I/O module → sends it to printer

Functions of I/O Module

3. Data Buffering → Temporary storage

- Devices are **slower than CPU**, so speed mismatch happens.
- I/O module stores data temporarily in a **buffer**

This helps:

- Prevent data loss
- Smooth data transfer

4. Error Detection → Checks errors

- I/O module checks if data transfer has **errors**
- If error occurs:
- It informs CPU
- Data can be resent

Example:

- If data is corrupted while sending to printer.
- I/O module detects and reports it.

Modes of Data Transfer

Modes of Data Transfer : define how data is transferred between:

- **CPU**
- **Memory**
- **I/O Devices**

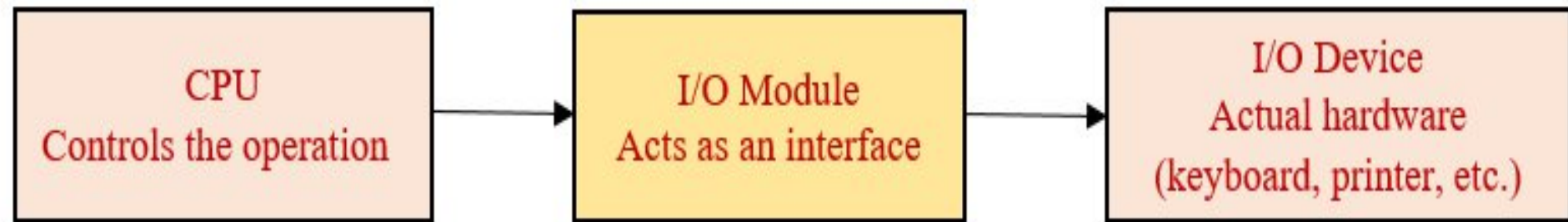
Types of Data Transfer Modes

There are **three main modes**:

- 1. Programmed I/O**
- 2. Interrupt-Driven I/O**
- 3. Direct Memory Access (DMA)**

Programmed I/O-

- In Programmed I/O, data transfer always happens through the **I/O module**, not directly between CPU and device.
- The CPU continuously checks whether an I/O device is ready and then performs the data transfer itself.



Components of I/O Module

The I/O module contains:

- **Data Register** → stores data temporarily
- **Status Register** → shows device status (ready/busy)
- **Control Register** → receives commands from CPU

Working of Programmed I/O

Step 1: Read Status Bit

- The program (CPU) **reads the status register** of the I/O module.
- It checks whether the device is **ready with data**.

This is done through the **status bit**:

- 1 = Ready
- 0 = Not Ready

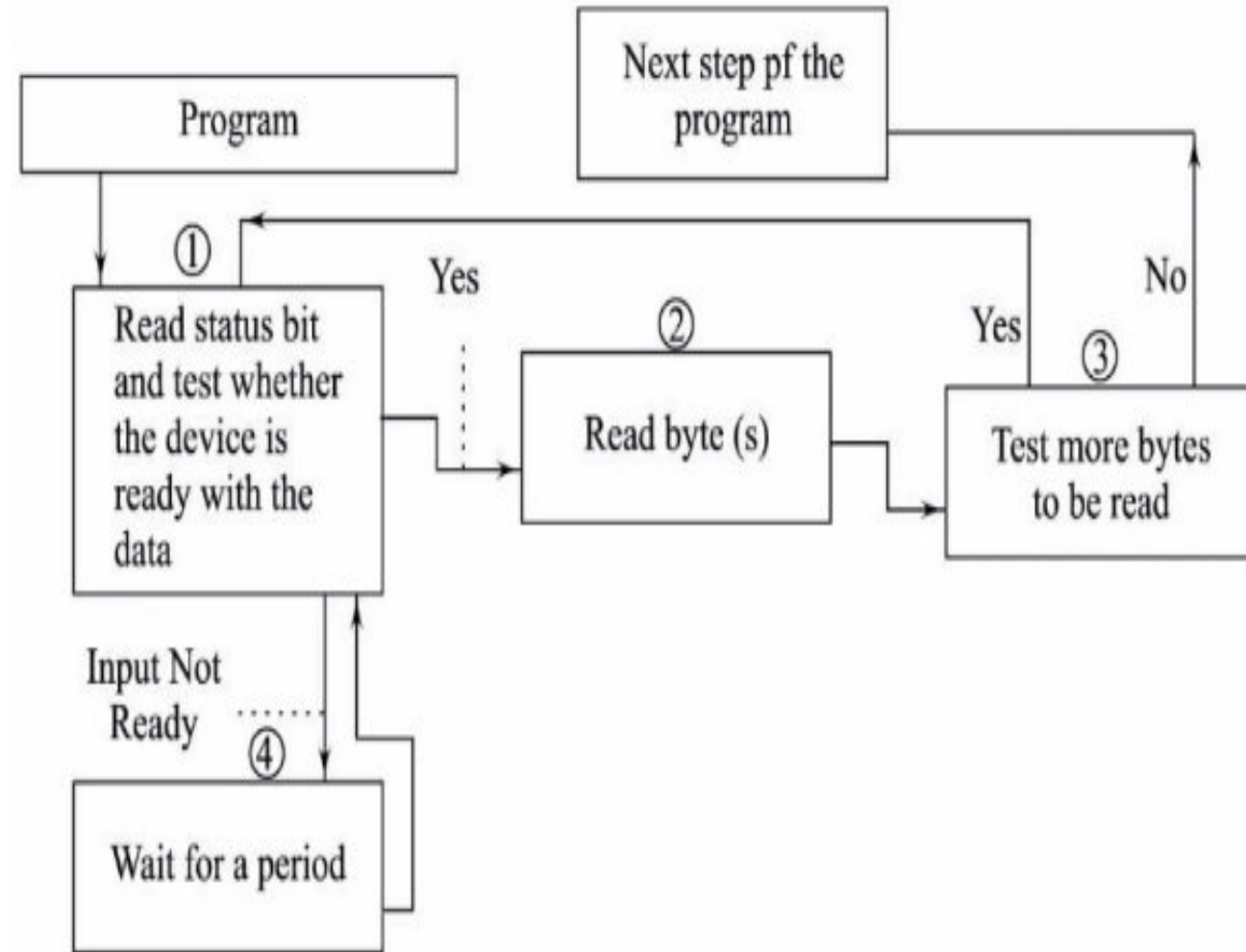
Step 2: Wait (if not ready)

If the device is **not ready**, the CPU:

- Waits for some time
- Again checks the status

This is called **busy waiting / polling**

CPU keeps looping here until the device becomes ready.



Working of Programmed I/O

Step 2: Read Byte(s)

- Once the device is **ready**, the CPU:
 - Reads the data from the **data register of the I/O module**
 - Transfers it to memory

Step 3: Test for More Data

After reading one byte:

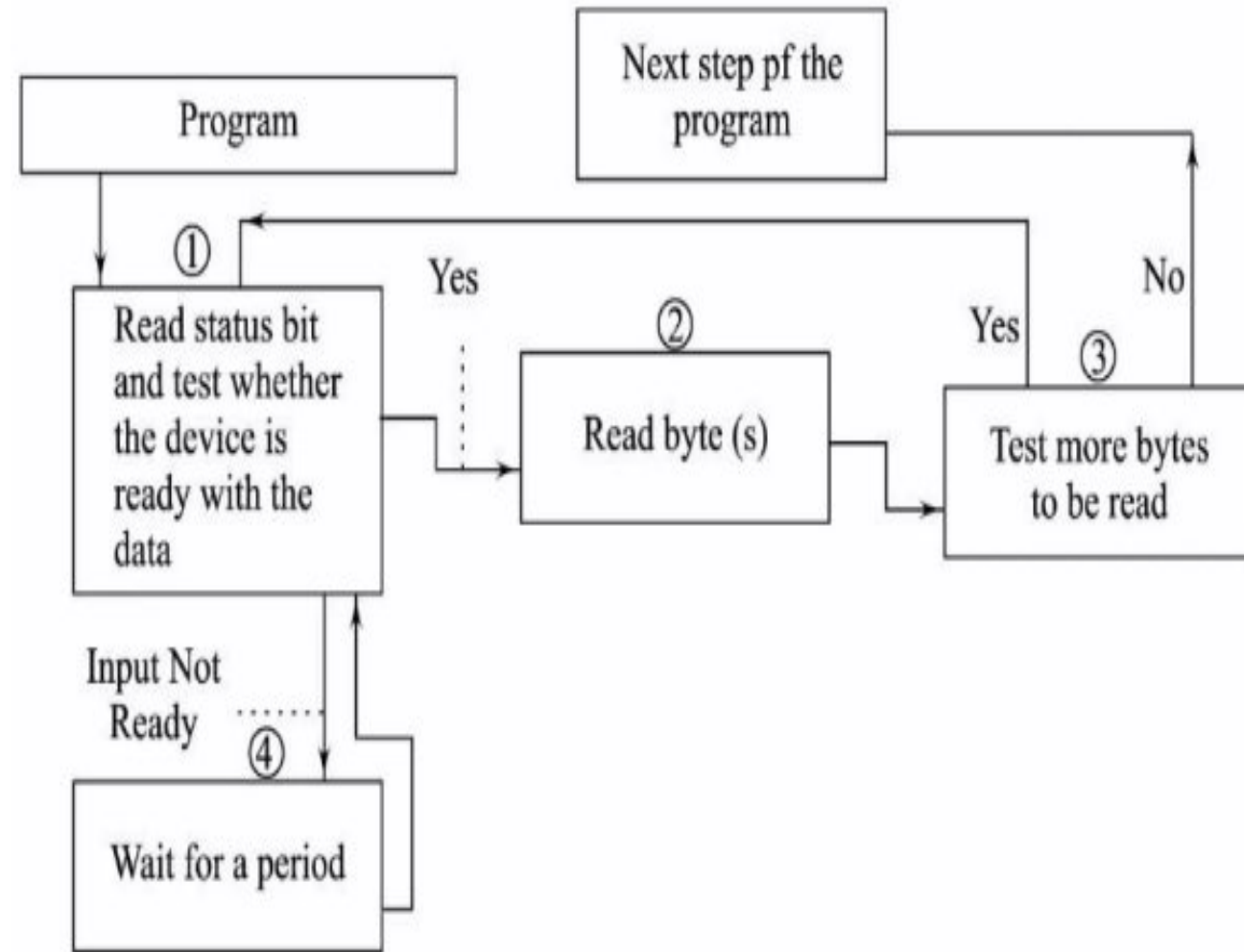
CPU checks if **more data is to be read**

If **YES**:

Go back to **Step 1** (check status again)

If **NO**:

Move to the **next step of the program**



Interrupt-Driven I/O

- Interrupt-Driven I/O is an I/O technique in which CPU does not continuously poll the device status.

Working

Step 1. CPU Initiates I/O Operation

- The CPU starts the process by giving an instruction like **READ** or **WRITE**.
- It sends this command to the **I/O module** (not directly to the device).

Internally:

Command is written into the **control register** of the I/O module.

CPU may also specify:

- Device address
- Type of operation (input/output)

Step 2: Command Sent to I/O Module

- The I/O module receives the command.
- It acts as a bridge between CPU and device.

Step 4. Device Becomes Ready

The I/O device takes some time (example: user pressing a key, printer getting ready).

Once ready:

Device informs the I/O module

Step 3: CPU Performs Other Tasks (No Polling)

- After giving the command, the CPU does NOT wait.
- It continues executing other instructions/programs.

Important: No busy waiting

Step 5. Interrupt Signal Generated

The I/O module sends an interrupt signal to the CPU.

Step 6: CPU Responds to Interrupt

CPU completes the current instruction

Then it:

- Temporarily stops current program
- Saves its current state (called context saving)

Step 7: Execution of ISR (Interrupt Service Routine)

- Identify which device sent the interrupt
- Read/write data via I/O module
- Data transfer happens here:
CPU ↔ I/O module ↔ Device

Step 8. Data Transfer Occurs

Data is transferred

Keyboard → I/O module → CPU →
Memory

Step 9: CPU Resumes Previous Task

After ISR is completed:

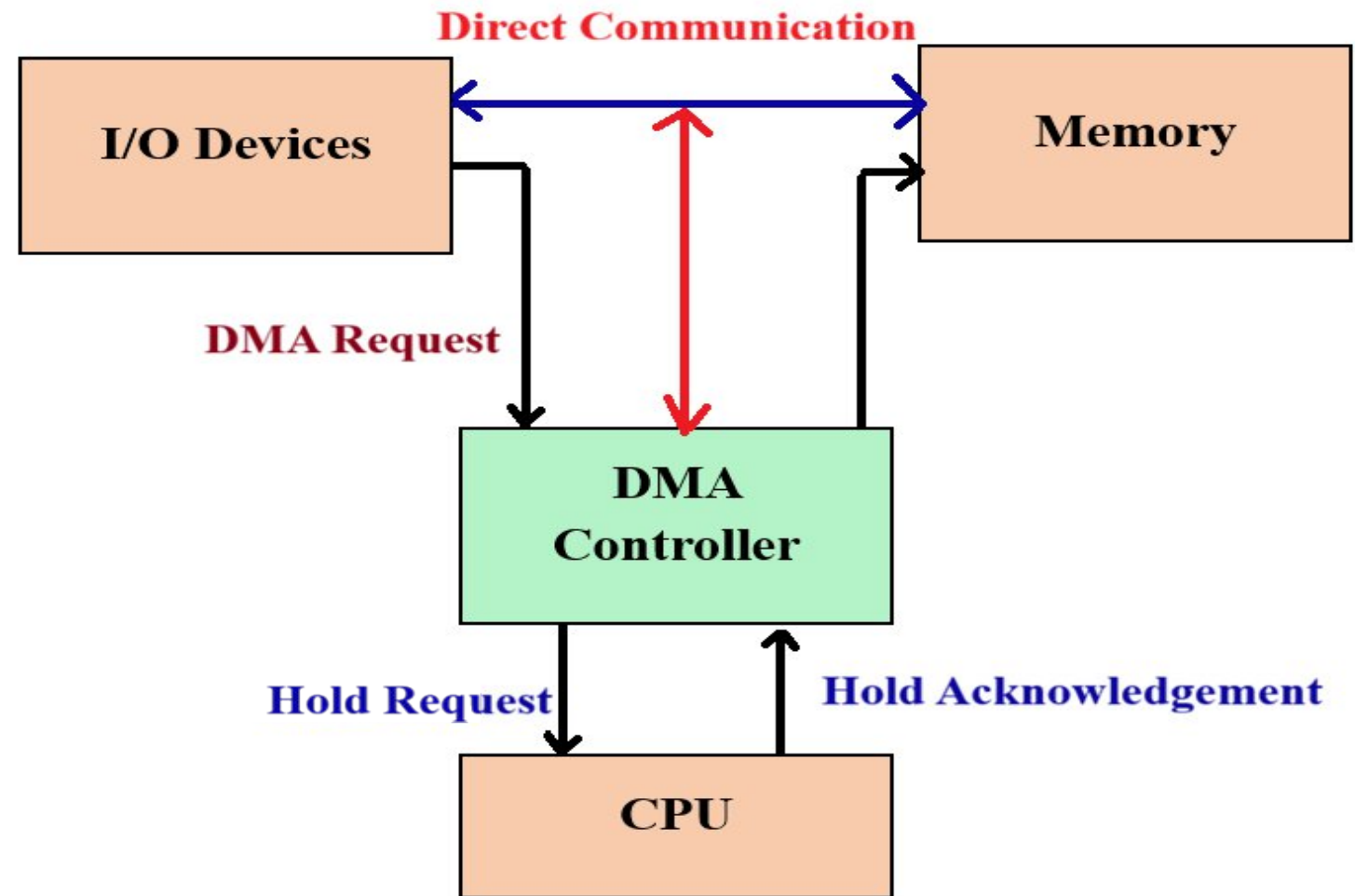
CPU restores saved state

Continues the interrupted program

DMA: (DMA) is a technology that allows hardware devices to transfer data between themselves and memory without involving the central processing unit (CPU).

A Direct Memory Access **(DMA) Controller** is a hardware unit in computers that enables peripheral devices (like hard drives, graphics cards, or network cards) to transfer data directly to or from main memory (RAM) without constant CPU involvement

i.e **I/O Devices to memory**



Working Of DMA

1. I/O device wants to send a data to memory. I/O device has to send DMA request to DMA Controller
2. DMA controller sends hold request to CPU.
3. DMA Controller waits until CPU sends hold acknowledgement to DMA.
4. CPU becomes Slave & DMA controller becomes master, DMA Controller takes control of the **system bus** .CPU leaves control over system bus.
5. DMA controller manages operation over system bus between CPU, Memory, I/O devices
6. Now I/o Devices can directly communicate with memory with the help of DMA controller
7. Completion Signal (Interrupt)
When transfer is finished: DMA sends an **interrupt signal** to CPU .
CPU understands: “Work is done”
8. CPU Resumes Control: CPU takes back control of system bus

Feature	Programmed I/O	Interrupt-Driven I/O	DMA
CPU Involvement	Very High	Medium	Very Low
Waiting	CPU waits (busy waiting)	No waiting	No waiting
Data Transfer	CPU ↔ I/O Module ↔ Device	CPU ↔ I/O Module ↔ Device	Device ↔ Memory (direct)
Efficiency	Low	Better	High
Speed	Slow	Moderate	Fast
CPU Utilization	Poor	Good	Excellent
Complexity	Simple	Moderate	High
Hardware Requirement	No extra hardware	Interrupt mechanism	DMA controller required

I/O Channels :

- An **I/O channel** is a **specialized hardware unit** that can **control I/O operations independently of the CPU**.
- It acts like a **mini-processor** dedicated only to I/O tasks.
- CPU gives instructions to the **I/O channel**
- Channel executes the entire I/O operation

Working of I/O Channel

1. CPU sends I/O instruction to the channel
2. Channel takes control of the I/O device
3. Channel manages:
 - Data transfer & Device control
4. CPU continues other work
5. Channel informs CPU after completion

I/O Processor (IOP):

- An **I/O Processor (IOP)** is a **more powerful and independent processor** designed specifically to manage I/O operations.
- It is like a **separate CPU for I/O tasks**
- IOP can execute its **own programs**.
- Works independently of the main CPU

Working of I/O Processor

1. CPU sends I/O task to IOP
2. IOP executes I/O instructions
3. Controls devices and manages data transfer
4. CPU performs other task.
5. IOP interrupts CPU after completion

Pipelining

Pipelining in computer architecture is a technique used to **increase CPU performance** by executing multiple instructions **simultaneously in different stages**.

- The CPU does **not wait** to finish one instruction completely.
- It **breaks each instruction into steps** and works on many instructions **at the same time**, each at a different step.

Advantages:

- Faster instruction execution (increased throughput)
- Better CPU utilization
- Improves overall system performance

Disadvantages:

- Design becomes complex
- Requires extra hardware, increasing cost and power.

Common Pipeline Stages : A typical instruction pipeline has 5 stages:

Fetch (IF) – Get instruction from memory

Decode (ID) – Understand the instruction

Execute (EX) – Perform operation (ALU work)

Memory (MEM) – Access memory if needed

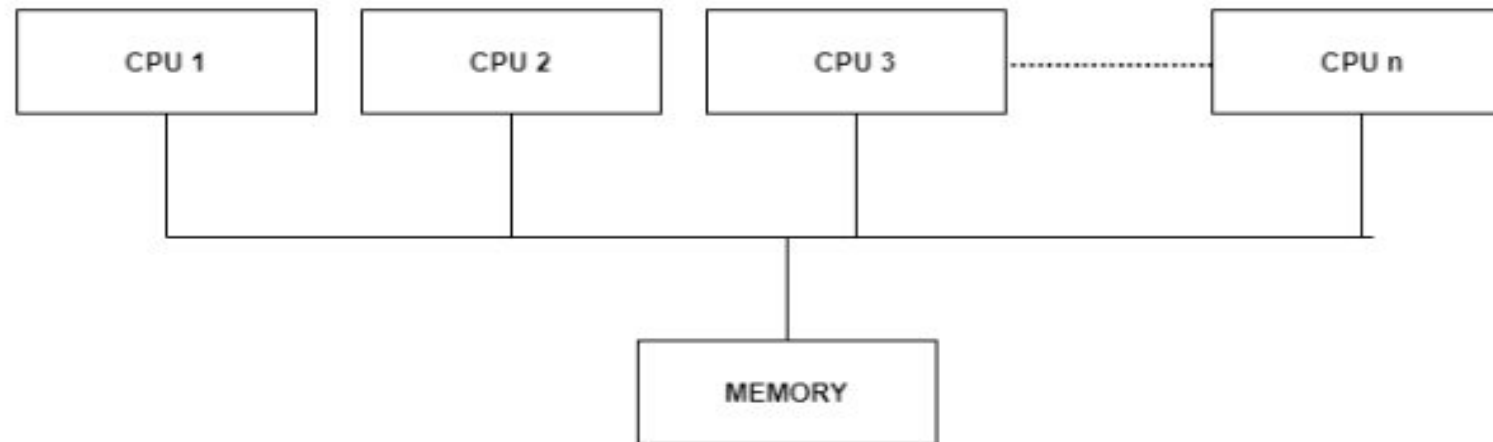
Write Back (WB) – Store result in register

Clock Cycle	Instruction 1	Instruction 2	Instruction 3
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX

Multiple instructions are processed in parallel

Multiple Processor Organization : Multiprocessing

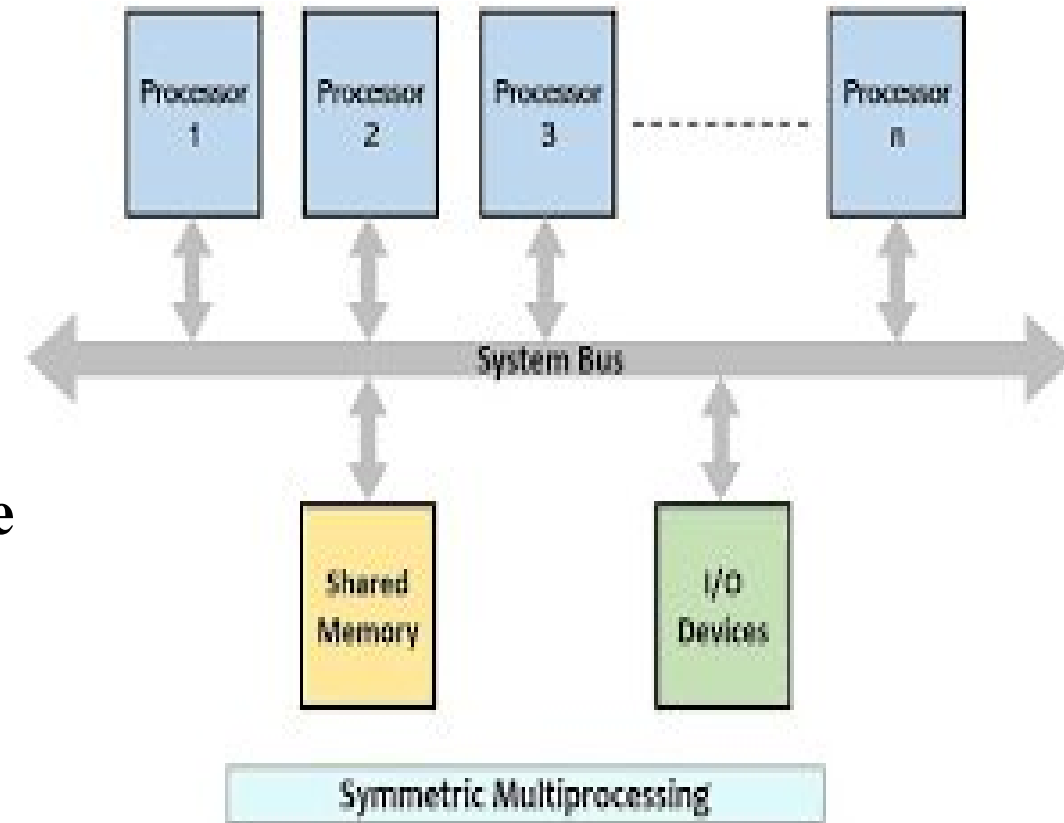
- **Parallel Processing (Multiple Processor Organization)** means using **more than one processor (CPU)** in a system to execute tasks **simultaneously**, which improves speed and performance.
- They are connected with physical memory, computer buses, clocks, and peripheral devices.
- The main objective of using a multiprocessor operating system is to increase the execution speed of the system.



1. Symmetric Multiprocessing: Shared Memory System (Tightly Coupled)
2. Asymmetric Multiprocessing: Distributed Memory System (Loosely Coupled)

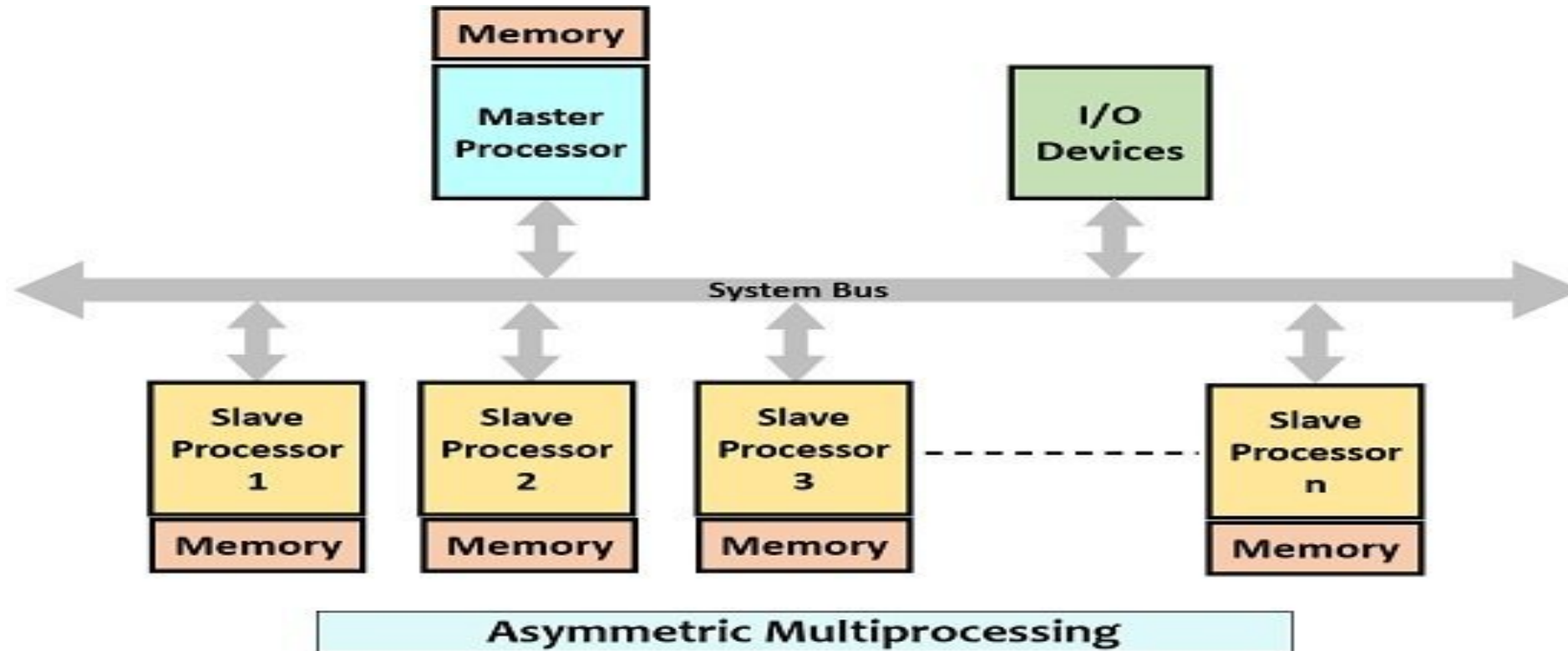
Shared Memory System (Tightly Coupled) works:

- All processors are connected to a **common main memory** through a shared bus or interconnection network.
- Each processor **fetches instructions and data** from the same memory.
- When one processor updates data, the change is **visible to all other processors** immediately.
- Processors communicate by **reading and writing shared variables** in memory .
- The operating system **manages task distribution** among processors.



How Distributed Memory System (Loosely Coupled) works:

- Each processor has its **own local memory** (no shared memory).
- A processor can **only directly access its own memory**.
- If it needs data from another processor, it must **send a message request**.
- The other processor **replies by sending the required data** back.
- Communication happens through a **network (like LAN/cluster interconnect)**.



Cache Coherence

- In multiprocessor systems, each CPU has its own **cache**.
- **Cache coherence** ensures that **all processors see the same correct value of data**, even if it is stored in multiple caches.

Problem:

- Processor 1 updates a value in its cache
- Processor 2 may still have the **old value** in its cache

Solution:

Cache coherence protocols keep all copies **consistent (updated)**.

MESI Protocol: MESI is a common cache coherence protocol with 4 states:

<u>M – Modified</u> • Data is changed in cache • Different from main memory • Only this cache has the latest value	<u>S – Shared</u> • Data is same as main memory • Present in multiple caches
<u>E – Exclusive</u> • Data is same as main memory • Present in only one cache	<u>I – Invalid</u> • Data is not valid (outdated) • Cannot be used

How it works :

- If one processor **modifies data (M)** → other caches mark it **Invalid (I)**
- If multiple processors read → data becomes **Shared (S)**
- Ensures all processors always use **correct data**